Offloading Code Efficiently on Mobile Cloud

Samia Tasnim
School of Computing and Information Sciences
Florida International University
Email: stasn002@fiu.edu

Kishwar Ahmed, Niki Pissinou, and S. S. Iyengar
Email: { kahme006, pissinou}@fiu.edu, iyengar@cis.fiu.

1. INTRODUCTION

Despite significant improvement in mobile technologies, the capacity of mobile devices are inadequate to satiate their hunger for resources (e.g., energy, CPU etc). With the increase of network resources namely high performance clouds, it is constantly investigated to find the optimal partitioning between the loads of mobile device and the cloud. Much of the recent research has been focused on different aspects of offloading. Along with offloading, the location of the servers used for computational benefits is also important. In many cases the transfer latency adversely affects the performance of the offloading mechanism. We investigate that, the communication latency between the device and the cloud, and the current service load of the cloud is a factor in making offloading decision.

In our work, we focus on the network latency and relative distance between the device and the service provider. Limiting the latency of communication is of main importance. We also consider the mobility of the mobile device, so that the device can communicate with the nearest possible server in terms of cost.

2. RELATED WORK

There are different ways of offloading discussed in past literature. In [2], the whole image of smart phone is transferred to the cloud, while according to [1], partial image is transferred keeping the rest inside the mobile for further computation. An extension of CloneCloud is [3], which has improved on the intermittent connectivity issue of CloneCloud. However, to the best of our knowledge, no work has been done on offloading of service code considering location change of the mobile device, and also depending on the partitioning of the services.

3. ARCHITECTURE

3.1 Location Change Decision Module

Whenever there is change of location of a Mobile Device(*MD*) in terms of location area, this module helps to take decision on whether to continue using the previous cloud server for service execution or to assign a new cloud server. VLR of the location area will keep a list of nearest cloud servers (**List$_c$**). The hand-off mechanism of current 3*G* networks will work as a trigger to our algorithm and send the list of nearest cloud servers to the *MD*.

Algorithm 1: Update Native Cloudlist

```
if List_n == List_c then
    No calculation required
else if List_n ⊂ List_c then
    S = List_c - List_n
    for all S_i ∈ S do
        Calculate gain factor for S_i
    end for
    CreateProfileTree()
else if Listc ⊂ Listn then
    List_n = List_c
else
    for all S_i ∈ List_c do
        Calculate gain factor for S_i
    end for
    List_n = List_c
    CreateProfileTree()
end if
```

List$_n$: Current list maintained in the MD for nearest cloud servers.

3.2 Profile Tree Building Module

We propose to build cost tree for different partitions of an application by comparing the current server capacity vector (SCV), with the mobile capacity vector (MCV), which results in reduced cost in terms of data transfer as well as immediate decision making. The ratio of SCV and MCV has been termed as gain factor. SCV denotes a set of constraints (e.g., CPU speed, memory etc) which determines the performance capability of the cloud server while MCV denotes a similar set of constraints for the MD.

3.3 Service Migration Decision Module

This module decide whether to transfer the service to the cloud based on service profile tree. To dynamically transfer a part of a service, we take into consideration migration cost(m_i) along with the service execution cost (E_i), which can be found from the profile tree built for each server [4].

4. SIMULATION SETUP

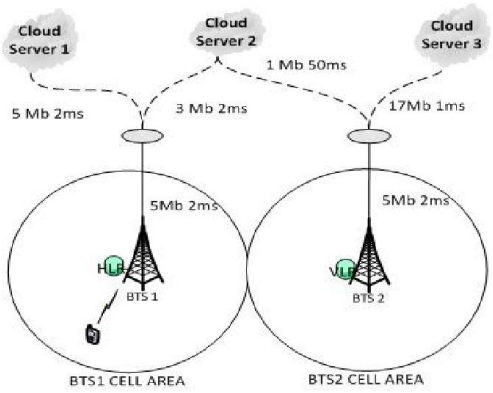

Figure 1: Simulation Scenario

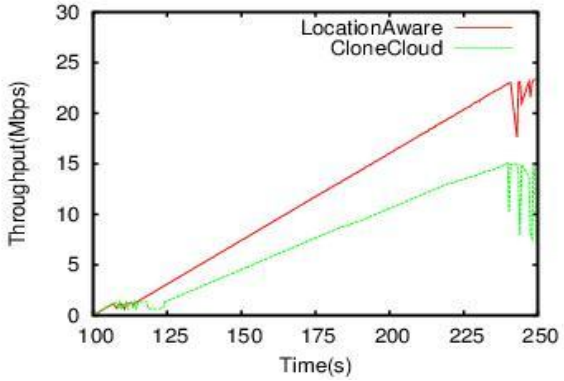

Figure 2. Throughput Comparison

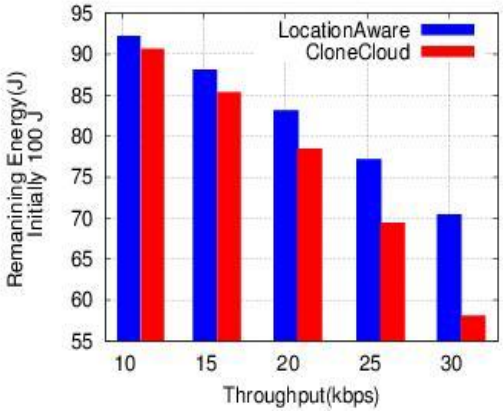

Figure 3. Remaining Energy Comparison for same throughput

5. PERFORMANCE EVALUATION

This section outlines the comparison of our algorithm with other algorithm (CloneCloud) based on the simulation setup we have used. We compare on the basis of the following comparison criteria:

5.1 Throughput

Figure 2 demonstrates the better throughput gained (approximately 50%) by our LocationAware algorithm in comparison to CloneCloud.

5.2 Power

It is evident from Figure 3 that LocationAware algorithm incurs less energy in comparison to CloneCloud to provide same amount of service.

6. CONCLUSION

Our proposed system incorporate the location change of the mobile device before code offloading to a cloud server. We avoid extra cost of dynamic profiling. Instead of a fixed cloud server, depending on the geolocation of the mobile device, as well as the existing load and capacity of the nearby cloud servers, the choice of the cloud server is made for code offload. The simulation results show how our algorithm outperforms the CloneCloud algorithm in perspective of throughput and energy consumption.

7. REFERENCES

[1] Chun, Byung-Gon, Sunghwan Ihm, Petros Maniatis, Mayur Naik, and Ashwin Patti. "Clonecloud: elastic execution between mobile device and cloud." In*Proceedings of the sixth conference on Computer systems*, pp. 301-314. ACM, 2011.

[2] Portokalidis, Georgios, Philip Homburg, Kostas Anagnostakis, and Herbert Bos. "Paranoid android: versatile protection for smartphones." In *Proceedings of the 26th Annual Computer Security Applications Conference*, pp. 347-356. ACM, 2010.

[3] Shi, Cong, Mostafa H. Ammar, Ellen W. Zegura, and Mayur Naik. "Computing in cirrus clouds: the challenge of intermittent connectivity." In *Proceedings of the first edition of the MCC workshop on Mobile cloud computing*, pp. 23-28. ACM, 2012.

[4] Tasnim, Samia, Mohammad Ataur Rahman Chowdhury, Kishwar Ahmed, Niki Pissinou, and S. S. Iyengar. "Location aware code offloading on mobile cloud with QoS constraint." In *Consumer Communications and Networking Conference (CCNC), 2014 IEEE 11th*, pp. 74-79. IEEE, 2014.